TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN 01

Môn: Học sâu

XẤP XỈ HÀM MẬT ĐỘ GAUSSIAN HAI CHIỀU BẰNG MẠNG PERCEPTRON ĐA TẦNG (MLP) TRONG PYTORCH

Nhóm thực hiện:

Lê Phan Nguyên Hải	3123411081
Võ Hoàng Bảo	3123411030
Đoàn Trung Kiên	3123411166
Huỳnh Viễn Thông	3123411287

Giảng viên: TS. Đỗ Như Tài

Thành phố Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

Chương 1. Giới thiệu 3
1.1. Mục tiêu 3
1.2. Phát biểu bài toán 3
1.3. Kết quả chính 3
1.4. Thành viên và đóng góp 4
Chương 2. Dữ liệu 4
2.1. Hàm mục tiêu 4
2.2. Lấy mẫu và chia dữ liệu 5
2.3. Thống kê và trực quan hóa 6
Chương 3. Mô hình và cài đặt 7
3.1. Kiến trúc MLP và lan truyền thuận 7
3.2. Hàm mất mát, lan truyền ngược và Adam 8
3.3. Cài đặt PyTorch và quy trình huấn luyện 9
3.4. Môi trường và tái lập 10
Chương 4. Thực nghiệm 11
4.1. Thiết lập và quy trình đánh giá công bằng 11
4.2. Thực nghiệm A: số mẫu huấn luyện 14
4.3. Thực nghiệm B: số nơ-ron của tầng ẩn 16
4.4. Thực nghiệm C: số tầng ẩn 20
4.5. Mô hình cuối và đánh giá trên tập test 22
Chương 5. Thảo luận 24
5.1. Vì sao không thấy đường chữ U của quá khớp 24
5.2. Vì sao mạng sâu tốt hơn mạng rộng 25
5.3. Những kết quả không như kỳ vọng 25
5.4. Hạn chế 26
5.5. Đề xuất cải tiến 26
Chương 6. Kết luận 26
Tài liệu tham khảo 27
Phụ lục A. Lệnh tái lập 28
Phụ lục B. Nhật ký quyết định 28

Chương 1. Giới thiệu

1.1. Mục tiêu

Bài tập lớn yêu cầu xây dựng và huấn luyện mạng perceptron đa tầng (MLP) bằng PyTorch để xấp xỉ một hàm số hai biến, rồi làm thực nghiệm có kiểm soát để trả lời ba câu hỏi: số mẫu huấn luyện (A), số nơ-ron của tầng ẩn (B) và số tầng ẩn (C) ảnh hưởng thế nào đến sai số. Ngoài việc chạy được mô hình, nhóm đặt mục tiêu mọi con số trong báo cáo phải truy vết được về file log, CSV hoặc hình trong repo, và tập test không dùng để chọn siêu tham số hay mô hình (các cột Test của A, B, C là hậu kiểm sau khi đã chọn xong).

1.2. Phát biểu bài toán

Cho hàm mật độ Gaussian hai chiều với $\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ và ma trận hiệp phương sai $C = \begin{bmatrix} 0.3 & -0.5 \\ -0.5 & 2 \end{bmatrix}$. Lấy ngẫu nhiên $x = (x_1, x_2)$ trong miền $-2 \le x_1 \le 2$, $-4 \le x_2 \le 4$ và tính nhãn $y = f(x_1, x_2)$. Cần xây dựng MLP hai đầu vào, một đầu ra $\hat{y}$ sao cho $\hat{y} \approx y$; có tập validation riêng để chọn mô hình và một tập test độc lập 100 mẫu. Đây là bài toán hồi quy, đánh giá bằng MSE cùng MAE, RMSE và R^2.

1.3. Kết quả chính

- Mạng sâu tốt hơn rõ rệt so với mạng nông: mô hình cuối [10,10,10] (261 tham số) đạt Test MSE 7.13e-8 ± 2.36e-8 và $R^2 = 0.99997$, thấp hơn khoảng 320 lần so với baseline một tầng [10].
- Ở cùng số tham số, mạng nhiều tầng tốt hơn mạng một tầng rộng 16–28 lần, nên lợi ích đến từ độ sâu chứ không chỉ từ số tham số.
- Với dữ liệu không nhiễu, nhóm không quan sát được đường chữ U của quá khớp, kể cả khi giảm còn A = 200 mẫu; chỉ thấy khoảng cách train–validation tăng dần. Chương 5 phân tích nguyên nhân.
- Một phần lỗi cao ở A nhỏ là do được huấn luyện ít bước cập nhật hơn, không chỉ do ít dữ liệu; nhóm tách hai yếu tố này bằng một thí nghiệm đối chứng.

1.4. Thành viên và đóng góp

Bảng 1.1. *Thành viên, nhiệm vụ và tỉ lệ đóng góp.*

TV	Họ tên	MSSV	Nhiệm vụ chính	Đóng góp
TV1	Đoàn Trung Kiên	3123411166	Sinh dữ liệu, EDA, khung repo, README; Thực nghiệm A và đối chứng cùng ngân sách bước; Ch.2, mục 4.2	25%
TV2	Lê Phan Nguyên Hải	3123411081	Lớp MLP, vòng huấn luyện, early stopping, checkpoint; chọn baseline, so sánh optimizer; Thực nghiệm C và đối chứng cùng số tham số; Ch.3, mục 4.4	25%
TV3	Huỳnh Viễn Thông	3123411287	Hàm đánh giá, thư viện vẽ hình; Thực nghiệm B (A = 1000 và A = 200); chọn mô hình cuối và đánh giá test; mục 4.3, 4.5	25%
TV4	Võ Hoàng Bảo	3123411030	Sơ đồ, bảng tự động, kiểm tra tái lập; mục 4.1, Ch.1, Ch.5, Ch.6; ghép báo cáo và slide	25%

Chương 2. Dữ liệu

2.1. Hàm mục tiêu

$$f_X(x) = \frac{1}{(2\pi)^{n/2}[\det(C_X)]^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_X)^T C_X^{-1}(x-\mu_X)\right)$$

$$\mu_X = \begin{bmatrix}\mu_1\\ \mu_2\end{bmatrix} = \begin{bmatrix}0\\ 0\end{bmatrix} \qquad \mathbf{C}_X = \begin{bmatrix}0.3 & -0.5\\ -0.5 & 2\end{bmatrix}$$

Với n = 2 (số chiều của x), ta có $\det(C) = 0.3 \cdot 2 - 0.5^2 = 0.35$, nên giá trị lớn nhất tại tâm là $y_{\max} = \frac{1}{2\pi\sqrt{0.35}} \approx 0.2690$. Nhãn vì vậy nằm trong $y \in (0; 0.269]$, không phải [0; 1]. Ma trận nghịch đảo là $C^{-1} = \frac{1}{0.35}\begin{bmatrix}2 & 0.5\\ 0.5 & 0.3\end{bmatrix} \approx \begin{bmatrix}5.714 & 1.429\\ 1.429 & 0.857\end{bmatrix}$. Khai triển dạng toàn phương trong hàm mũ:

$$Q(x) = (x-\mu)^T C^{-1}(x-\mu) = 5.714 \cdot x_1^2 + 2.857 \cdot x_1 x_2 + 0.857 \cdot x_2^2$$

Hệ số của x_1^2 lớn hơn x_2^2 khoảng 6.7 lần, nên hàm giảm nhanh theo x_1 và chậm theo x_2: "đồi" Gaussian hẹp theo x_1, dài theo x_2. Số hạng chéo $2.857 \cdot x_1 x_2$ dương (do hiệp

phương sai −0.5 âm) làm Q nhỏ hơn khi x_1 và x_2 trái dấu, tức đồi nghiêng về phía góc phần tư II và IV. Hệ số tương quan $\rho = \frac{-0.5}{\sqrt{0.3 \cdot 2}} \approx -0.645$. Trị riêng của C là 2.136 và 0.164; vectơ riêng ứng với trị riêng lớn là (−0.263, 0.965), lệch khoảng 15° so với trục x_2. Bảng 2.1 tính tay vài điểm để kiểm tra hàm cài đặt trong src/data/gaussian.py.

Bảng 2.1. *Giá trị f tại một số điểm (tính bằng gaussian_pdf, khớp công thức trên).*

(x_1, x_2)	Q(x)	f(x)	Nhận xét
(0, 0)	0	0.2690	Đỉnh, bằng y_max
(1, 0)	5.714	0.0155	Ra xa 1 đơn vị theo x_1: còn 5.7% đỉnh
(0, 2)	3.429	0.0484	Ra xa 2 đơn vị theo x_2 vẫn còn 18% đỉnh
(−0.5, 1)	0.857	0.1753	Trái dấu: nằm dọc trục chính của đồi
(0.5, 1)	3.714	0.0420	Cùng dấu: cùng khoảng cách nhưng thấp hơn 4 lần

Miền lấy mẫu [−2, 2] × [−4, 4] chứa 99.52% thể tích của mật độ (tích phân số trên lưới), nên gần như toàn bộ phần có giá trị của hàm nằm trong miền.

2.2. Lấy mẫu và chia dữ liệu

Mỗi điểm x được lấy đều và độc lập trên hình chữ nhật, nhãn tính trực tiếp từ f, không thêm nhiễu. Nhóm chọn lấy mẫu đều (không lấy theo phân phối Gaussian) vì mục tiêu là xấp xỉ hàm trên toàn miền, kể cả vùng đuôi. Ba tập được sinh độc lập bằng numpy.random.default_rng với seed riêng (Bảng 2.2); script src/generate_data.py ghi ba file CSV và data/meta.json. Các mức A của Thực nghiệm A lấy A mẫu đầu tiên của train pool, nên các tập train lồng nhau: tăng A chỉ thêm mẫu mới, không thay mẫu cũ.

Bảng 2.2. *Chia dữ liệu.*

Tập	Số mẫu	Seed	Vai trò
Train pool	2000	2026	Lấy tập con A = 200, 400, …, 2000
Validation	200	2027	Early stopping, chọn checkpoint và chọn cấu hình
Test	100	2028	Chỉ đánh giá mô hình cuối, sau khi đã chọn xong bằng validation

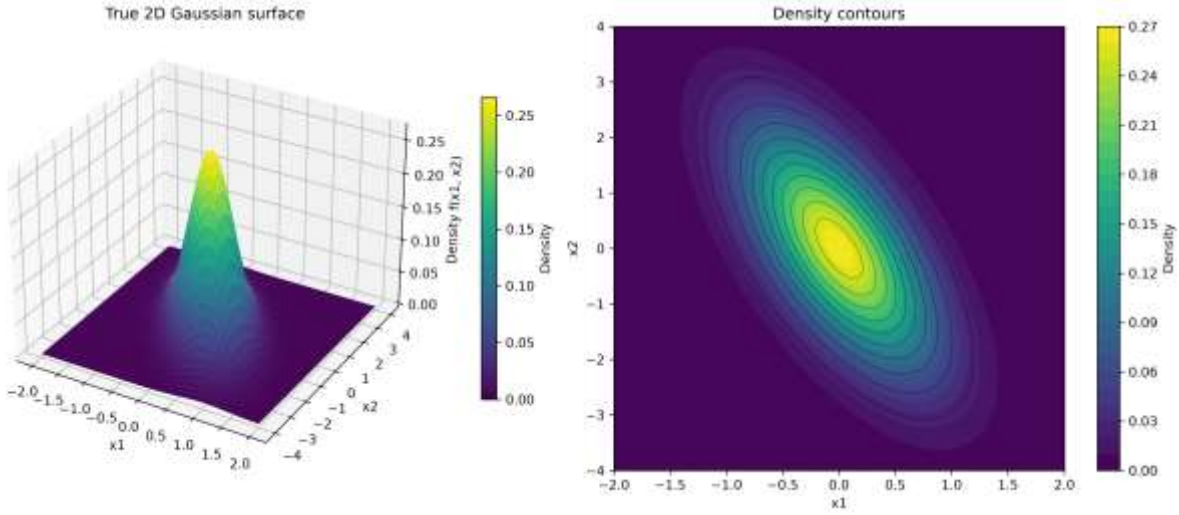
Tiền xử lý. Đầu vào chia cho biên miền $\left(\frac{x_1}{2}, \frac{x_2}{4}\right)$ để về $[-1, 1]$; nhãn chia cho y_max ≈ 0.2690 để về [0, 1]. Đây là lựa chọn của nhóm, không phải yêu cầu của đề. Các hằng số này lấy từ đề bài, không ước lượng từ dữ liệu, nên không dùng thông tin của validation hay test. Mọi chỉ số trong báo cáo được tính trên thang y gốc.

2.3. Thống kê và trực quan hóa

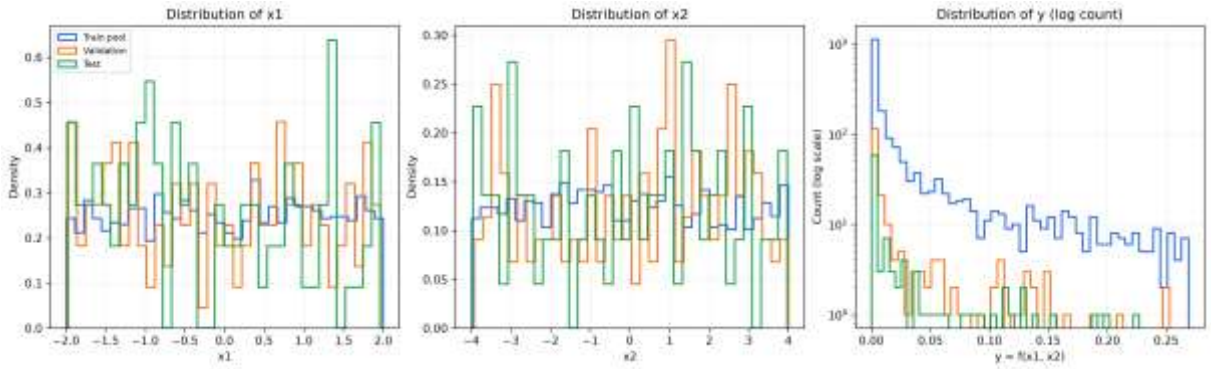
Bảng 2.3. *Thống kê mô tả của nhãn y (nguồn data/meta.json, results/eda/data_summary.csv).*

Tập	Min	Max	Mean	Std	Tỉ lệ y < 0.01
Train pool	8.9e-14	0.2687	0.0314	0.0581	62.6%
Validation	1.1e-11	0.2526	0.0264	0.0497	66.0%
Test	4.5e-13	0.2269	0.0268	0.0485	61.0%

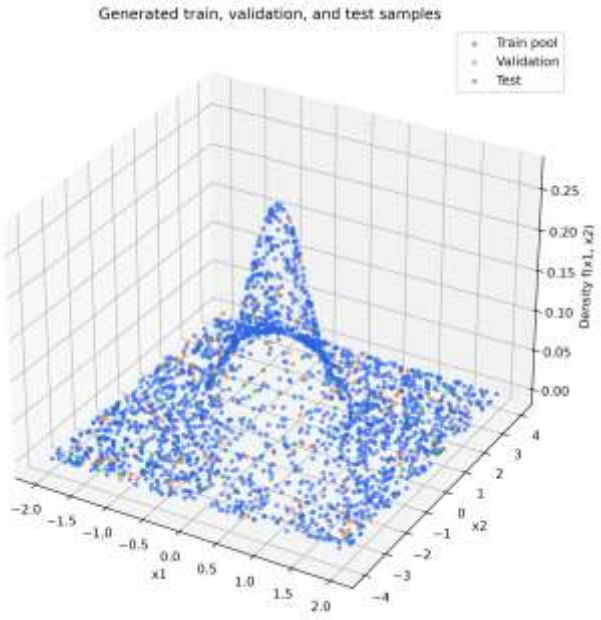
x_1 và x_2 phủ gần hết miền ở cả ba tập (min/max sát biên, độ lệch chuẩn gần $\frac{4}{\sqrt{12}} \approx 1.15$ và $\frac{8}{\sqrt{12}} \approx 2.31$ của phân phối đều). Nhãn mất cân bằng mạnh: 62.6% mẫu train có y < 0.01, đỉnh chỉ xuất hiện ở một vùng hẹp quanh tâm. Hệ quả là MSE tuyệt đối rất nhỏ ngay cả với mô hình kém; mô hình luôn đoán bằng trung bình đã có validation MSE 2.49e-3. Vì vậy nhóm luôn đặt mô hình hằng số vào bảng làm mốc và báo cáo thêm R^2, sai số trên lưới đều và bản đồ sai số.



Hình 2.1. *Bề mặt Gaussian thật (trái) và đường đồng mức (phải) trên miền lấy mẫu.*



Hình 2.2. *Histogram của x_1, x_2 và y; trục đếm của y theo thang log để thấy các bin thưa ở vùng đỉnh.*



Hình 2.3. *Scatter 3D của train pool, validation và test.*

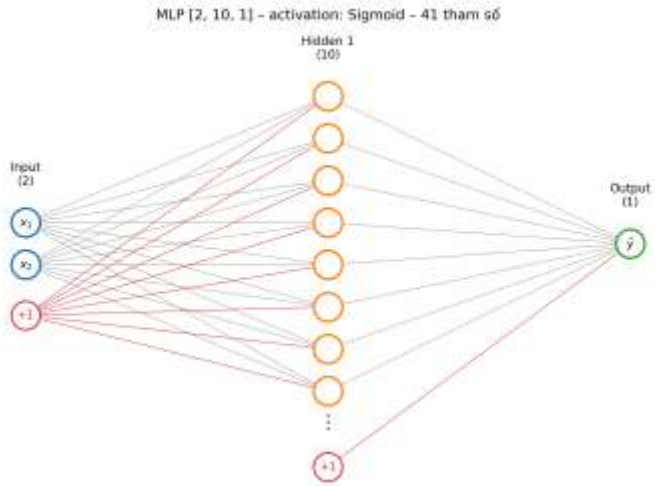
Chương 3. Mô hình và cài đặt

3.1. Kiến trúc MLP và lan truyền thuận

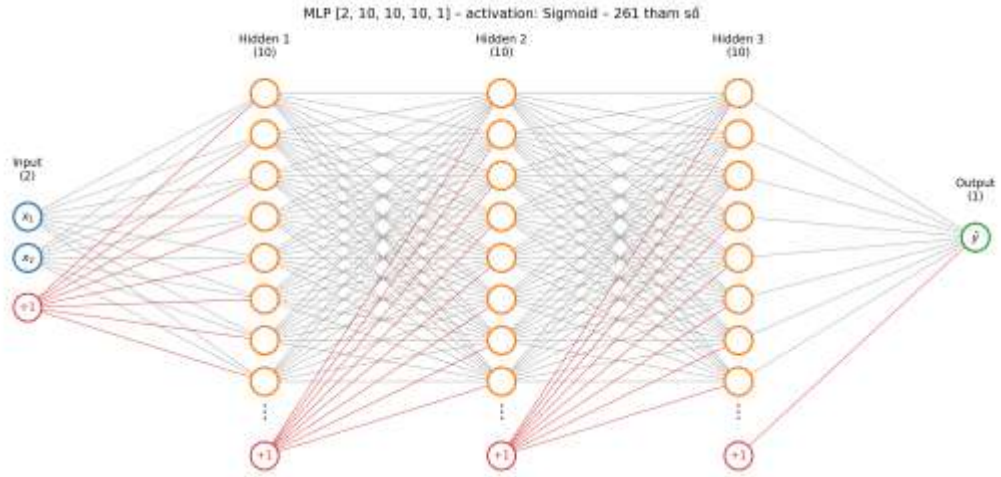
Lớp MLP (src/model/mlp.py) nhận danh sách số nơ-ron từng tầng ẩn hidden_sizes và tên hàm kích hoạt, nên một lớp dùng chung cho cả ba thực nghiệm: A giữ [10], B đổi [B], C đổi [10]×C. Tầng ra tuyến tính vì đây là hồi quy.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma\left(z^{(l)}\right), \quad a^{(0)} = \left(\frac{x_1}{2}, \frac{x_2}{4}\right), \quad \hat{y} = W^{(L)}a^{(L-1)} + b^{(L)}$$

với $\sigma(z) = \frac{1}{1+e^{-z}}$. Mạng một tầng [2, B, 1] có 4B + 1 tham số; mạng C tầng, mỗi tầng 10 nơ-ron, có 41, 151, 261, 371 tham số với C = 1…4. Hàm n_params đếm trực tiếp từ model và được kiểm thử khớp công thức.



Hình 3.1. *Kiến trúc baseline [2, 10, 1], sigmoid, 41 tham số (vẽ từ model thật bằng src/visual/draw_mlp.py).*



Hình 3.2. *Mạng ba tầng ẩn [10, 10, 10] (261 tham số), là mô hình được chọn cuối cùng.*

```
ACTS = {"tanh": nn.Tanh, "sigmoid": nn.Sigmoid, "relu": nn.ReLU}

class MLP(nn.Module):
    def __init__(self, hidden_sizes=(10,), activation="tanh"):
        super().__init__()
        layers, d = [], 2
        for h in hidden_sizes:
            layers += [nn.Linear(d, h), ACTS[activation]()]
            d = h
        layers.append(nn.Linear(d, 1))          # đầu ra tuyến tính (hồi quy)
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x).squeeze(-1)          # (N,) cùng shape với y
```

Baseline gọi MLP([10], activation="sigmoid"); giá trị mặc định "tanh" trong đoạn mã trên chỉ là mặc định của tham số.

3.2. Hàm mất mát, lan truyền ngược và Adam

Hàm mất mát là MSE trên một batch N mẫu, $L = \frac{1}{N}\sum_{n=1}^{N}(\hat{y}_n - y_n)^2$. Gradient lan ngược từ tầng ra [2]:

$$\delta^{(L)} = \frac{2}{N}(\hat{y} - y),\ \delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \odot \sigma'\left(z^{(l)}\right),\quad \frac{\partial L}{\partial W^{(l)}} = \delta^{(l)}\left(a^{(l-1)}\right)^T, \frac{\partial L}{\partial b^{(l)}} = \sum_n \delta_n^{(l)}$$

với σ′(z) = σ(z)(1 − σ(z)) ≤ 0.25. Mỗi tầng sigmoid nhân gradient với một hệ số không quá 0.25, nên mạng sâu có nguy cơ vanishing gradient [4]; mục 4.4 đo chuẩn gradient từng tầng để kiểm tra bằng số liệu. PyTorch [1] tính gradient bằng autograd; tests/test_backprop.py so autograd với lời giải tay bằng NumPy cho mạng 2→2→1 (sai khác < 10^{-12}) và với đạo hàm số qua torch.autograd.gradcheck. Adam [3] cập nhật tham số theo

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

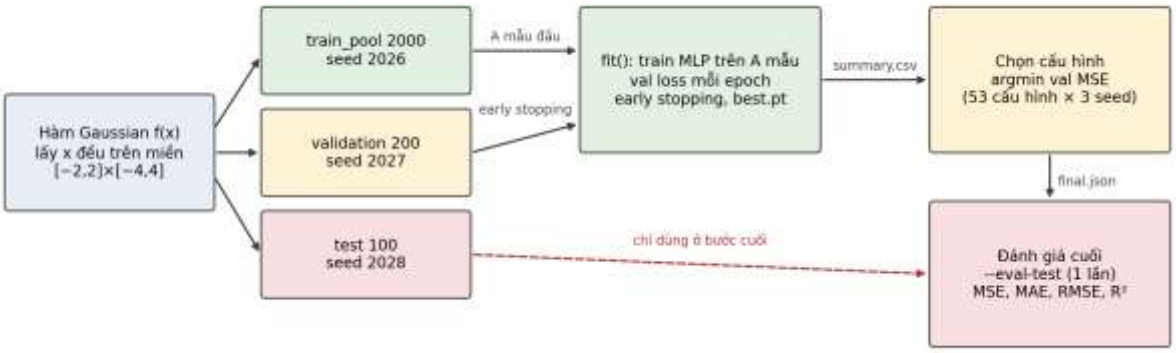
với m̂, v̂ là các ước lượng đã hiệu chỉnh độ lệch, β = (0.9, 0.999), ε = 10^{-8} (mặc định PyTorch) và η = 0.01.

3.3. Cài đặt PyTorch và quy trình huấn luyện

Hình 3.3 tóm tắt quy trình. make_loader (src/data/dataset.py) scale dữ liệu, gói vào TensorDataset và DataLoader batch 32; loader train xáo trộn bằng torch.Generator gắn seed nên thứ tự batch tái lập được. Vòng lặp một epoch theo đúng thứ tự đề yêu cầu:

```
def train_one_epoch(model, loader, opt, crit):
    model.train()
    tot = 0.0
    for xb, yb in loader:
        opt.zero_grad()
        pred = model(xb)
        assert pred.shape == yb.shape      # (N,1) với (N,) sẽ broadcast thành (N,N)
        loss = crit(pred, yb)
        loss.backward()
        opt.step()
        tot += loss.item() * xb.size(0)    # batch cuối có thể nhỏ hơn 32
    return tot / len(loader.dataset)
```

- validate chạy trong model.eval() và @torch.no_grad(), không xây đồ thị tính toán.
- forward trả shape (N,) và có assert shape: nếu dự đoán (N,1) còn nhãn (N,), MSELoss broadcast thành ma trận N×N, loss vẫn giảm nhưng mô hình học ra hằng số.
- fit sau mỗi epoch tính val loss; khi val tốt hơn mức tốt nhất thì lưu copy.deepcopy(model.state_dict()) (state_dict là tham chiếu, không deepcopy thì bản "tốt nhất" bị ghi đè). Nếu val không cải thiện sau patience = 200 epoch thì dừng, nạp lại trọng số tốt nhất và ghi best.pt. fit không nhận dữ liệu test.
- Mỗi run lưu config.json, history.csv (train loss, val loss, chuẩn gradient từng lớp theo epoch), metrics.json, best.pt trong results/<thí nghiệm>/<giá trị>/seed_<k>/; run_experiment.py gom lại thành summary.csv. Test chỉ được tính khi chạy với cờ --eval-test. Riêng results/grid_optimizer chỉ giữ thư mục run (history.csv, không có best.pt) của lr tốt nhất mỗi optimizer; đủ 126 dòng kết quả trong summary.csv.



Hình 3.3. *Quy trình train/validation/test. Test chỉ đi vào bước đánh giá cuối.*

Bảng 3.1. *Tổ chức mã nguồn.*

File	Chức năng
src/data/gaussian.py, src/generate_data.py	Hàm Gaussian, hằng số bài toán, sinh và lưu ba tập dữ liệu
src/data/dataset.py, src/data/eda.py	Đọc CSV, scale, DataLoader, tập con lồng nhau; thống kê và hình EDA
src/model/mlp.py	Lớp MLP, đếm tham số
src/engine/trainer.py, src/engine/evaluate.py	Vòng train/validate, early stopping, checkpoint; MSE, MAE, RMSE, R^2, dự đoán trên lưới
src/run_experiment.py, configs/*.json	Chạy một thí nghiệm theo config, lặp qua giá trị × seed, ghi summary.csv
src/posthoc_test.py, src/make_report_assets.py	Hậu kiểm test trên best.pt đã đóng băng; chọn mô hình cuối, đánh giá test và lưới
src/summarize_*.py, src/analyze_stopping.py, src/make_tables.py	Tổng hợp, vẽ hình, phân tích dừng sớm, sinh bảng report/tables/
tests/	Kiểm thử hàm Gaussian, dữ liệu, backprop, trainer, và config khớp log

3.4. Môi trường và tái lập

Mã chạy trên CPU: Windows 11, Python 3.12.10, PyTorch 2.6.0, NumPy 2.5.2, Pandas 3.0.3, Matplotlib 3.10.1 (results/env.txt). Mạng chỉ có 5–373 tham số nên CPU là đủ. set_seed cố định seed của random, NumPy và PyTorch; torch.set_num_threads(1) cho mọi run để thời gian so sánh được. Trên cùng máy, chạy lại cùng cấu hình cho ra cùng số đến từng chữ số (run A = 1000 của Thực nghiệm A trùng run sigmoid/lr 0.01 của lưới baseline); giữa hai máy, validation MSE lệch tương đối dưới 0.01%. Lệnh tái lập ở Phụ lục A.

Chương 4. Thực nghiệm

4.1. Thiết lập và quy trình đánh giá công bằng

Nguyên tắc. (1) Mỗi nhóm thực nghiệm chỉ thay đổi một yếu tố so với baseline (Bảng 4.1). (2) Mỗi cấu hình chạy 3 seed (0, 1, 2), khác nhau ở khởi tạo trọng số và thứ tự batch; dữ liệu giữ nguyên; báo cáo trung bình ± độ lệch chuẩn mẫu. (3) Mọi lựa chọn (siêu tham số, checkpoint, cấu hình cuối) chỉ dựa trên validation. Cột test của A, B, C là hậu kiểm: src/posthoc_test.py nạp best.pt đã đóng băng và đo trên test sau khi mọi lựa chọn đã xong, chỉ để trình bày đủ ba đường train/validation/test. (4) tests/test_config_evidence.py kiểm tra configs/baseline.json và configs/final.json khớp với cấu hình có val MSE thấp nhất trong log.

Bảng 4.1. *Siêu tham số baseline và cách chọn. "Chọn trước" nghĩa là chọn theo thông lệ, sau đó mới chạy thí nghiệm kiểm chứng.*

Siêu tham số	Giá trị	Cách chọn	Minh chứng
Kiến trúc	[2, 10, 1], đầu ra tuyến tính	Đề bài	—
Hàm kích hoạt, learning rate	sigmoid, 0.01	Khảo sát 3 × 3 rồi chọn	Bảng 4.2
Optimizer	Adam, β = (0.9, 0.999)	Chọn trước, kiểm chứng sau	Bảng 4.3
Batch size	32	Chọn trước, kiểm chứng sau	Bảng 4.4
Patience early stopping	200 epoch theo val loss	Chọn trước, kiểm chứng sau	Hình 4.2
Số epoch tối đa	2000	Chọn trước; là hạn chế (mục 5.4)	results/stopping/epoch_cap.csv
Số mẫu train	A = 1000 (trừ Thực nghiệm A)	Giữa lưới A	—
Seed	huấn luyện 0, 1, 2; dữ liệu 2026/2027/2028	Chọn trước	data/meta.json
Khởi tạo trọng số	mặc định nn.Linear	Mặc định PyTorch	—

Chọn hàm kích hoạt và learning rate. Lưới {tanh, sigmoid, relu} × {0.001, 0.003, 0.01} × 3 seed (27 run), mạng [10], Adam, A = 1000. Sigmoid với lr 0.01 có val MSE trung bình thấp nhất (Bảng 4.2); tanh với lr 0.003 đứng thứ hai nhưng khoảng ± std của hai cấu hình chồng nhau, nên đây là lựa chọn theo trung bình, chưa chứng minh sigmoid tốt hơn tanh. ReLU kém nhất ở mọi lr: mạng 10 nơ-ron ReLU chỉ tạo được xấp xỉ tuyến tính từng khúc rất thô cho một bề mặt cong.

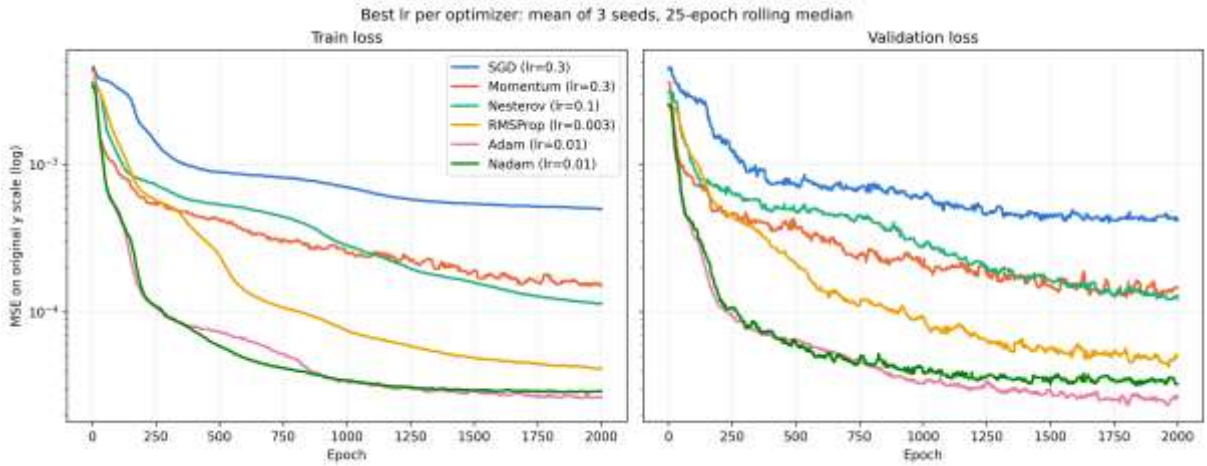
Bảng 4.2. *Lưới chọn hàm kích hoạt và learning rate (27 run, chỉ xét validation). Nguồn results/grid_baseline/summary.csv.*

lr_activation	Val MSE	Val MAE	Epoch tốt nhất
0.01_sigmoid	1.89e-5 ± 5.9e-6	3.35e-3 ± 5.6e-4	1653 ± 411
0.003_tanh	2.96e-5 ± 1.1e-5	4.19e-3 ± 8.5e-4	1663 ± 463
0.003_sigmoid	3.75e-5 ± 1.8e-5	4.62e-3 ± 1.3e-3	1989 ± 9
0.01_tanh	3.86e-5 ± 2.6e-5	4.48e-3 ± 1.7e-3	655 ± 194
0.001_tanh	4.16e-5 ± 1.3e-5	4.89e-3 ± 7.6e-4	1991 ± 14
0.001_sigmoid	1.17e-4 ± 2.8e-5	8.19e-3 ± 1.2e-3	1997 ± 4
0.003_relu	3.30e-4 ± 3.3e-4	1.19e-2 ± 6.5e-3	146 ± 128
0.001_relu	3.35e-4 ± 3.4e-4	1.18e-2 ± 6.4e-3	376 ± 347
0.01_relu	3.47e-4 ± 2.9e-4	1.26e-2 ± 5.3e-3	245 ± 234

Kiểm chứng optimizer. Adam được chọn trước (lưới trên đã dùng Adam), sau đó nhóm so sánh 6 optimizer (Momentum và Nesterov [8], RMSProp [9], Nadam [10]), mỗi optimizer dò riêng 7 lr từ 0.001 đến 1.0 (126 run, tắt early stopping để chạy đủ 2000 epoch). Adam ở lr 0.01 vẫn tốt nhất (Bảng 4.3, Hình 4.1), nên baseline giữ nguyên. Khoảng ± std của Adam, Nadam và RMSProp chồng nhau; Momentum và Nesterov kém Adam 4–5 lần, SGD kém khoảng 19 lần. Epoch tốt nhất của cả 6 optimizer nằm gần trần 2000 epoch (trung bình 1702–1961, Bảng 4.3), nên thứ hạng chỉ đúng với ngân sách 2000 epoch và chưa phản ánh mức hội tụ.

Bảng 4.3. *Kết quả ở lr tốt nhất của từng optimizer (126 run). Nguồn results/grid_optimizer/summary.csv.*

Optimizer_lr	Val MSE	Epoch tốt nhất
adam_0.01	1.88e-5 ± 5.9e-6	1906 ± 144
nadam_0.01	2.51e-5 ± 1.2e-5	1702 ± 445
rmsprop_0.003	3.38e-5 ± 2.1e-5	1961 ± 14
momentum_0.3	8.35e-5 ± 6.1e-5	1929 ± 65
nesterov_0.1	1.01e-4 ± 3.8e-5	1949 ± 62
sgd_0.3	3.57e-4 ± 7.7e-5	1838 ± 176

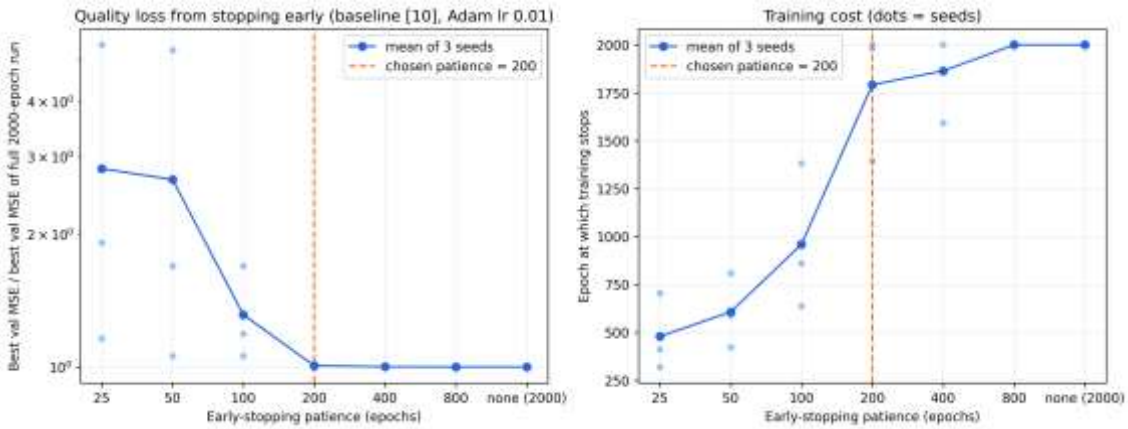


Hình 4.1. *Train loss và val loss theo epoch ở lr tốt nhất của mỗi optimizer (trung bình 3 seed, trung vị trượt 25 epoch).*

Kiểm chứng batch size và patience. Batch 32 có val MSE thấp nhất trong {16, 32, 64, 128} (Bảng 4.4), nhưng chênh lệch nhỏ hơn độ lệch chuẩn nên chỉ có thể nói batch 32 không kém các lựa chọn khác. Ngoài ra, batch 128 chạm trần 2000 epoch ở cả 3 seed (epoch tốt nhất 1962–1966) và batch 64 ở 2/3 seed; cùng trần epoch thì batch lớn có ít bước cập nhật hơn, nên phép so sánh này có cùng yếu tố gây nhiễu như mục 4.2. Để kiểm chứng patience (dừng sớm [11]), src/analyze_stopping.py mô phỏng các mức patience trên đường val của run Adam lr 0.01 chạy đủ 2000 epoch (Hình 4.2): so với không dừng sớm, patience 200 chỉ kém 0.7% val MSE, còn patience 100 kém 31%.

Bảng 4.4. *Kiểm chứng batch size trên baseline (12 run). Nguồn results/grid_batch/summary.csv.*

Batch	Val MSE	Epoch tốt nhất	Số epoch chạy
16	2.00e-5 ± 4.0e-6	1003 ± 177	1203 ± 177
32	1.89e-5 ± 5.9e-6	1653 ± 411	1792 ± 346
64	2.38e-5 ± 8.3e-6	1653 ± 537	1745 ± 442
128	2.27e-5 ± 9.0e-6	1965 ± 2	2000



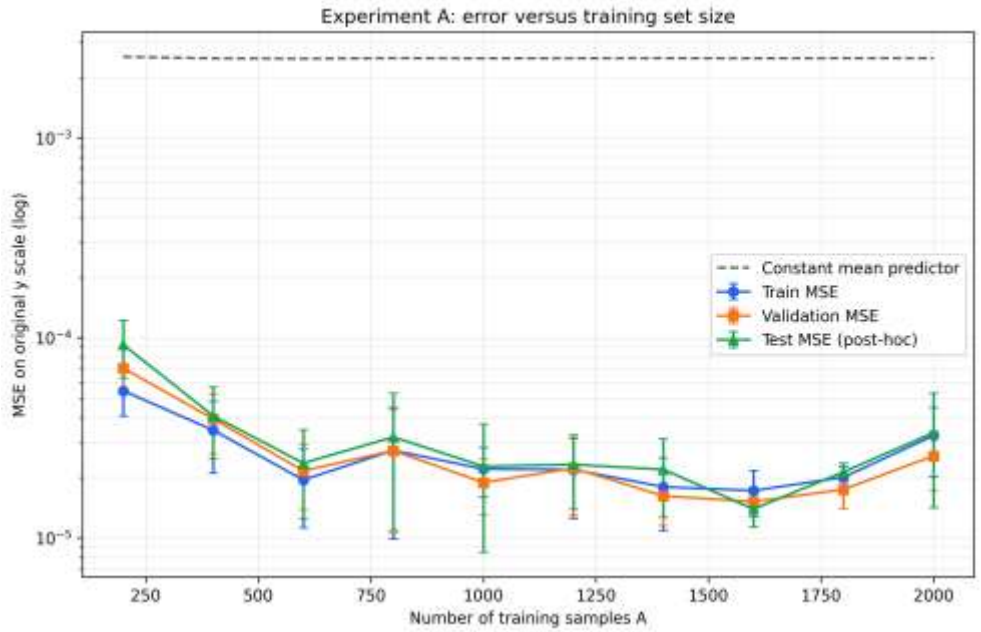
Hình 4.2. *Mô phỏng patience trên đường val chạy đủ 2000 epoch: val MSE tương đối so với không dừng sớm.*

4.2. Thực nghiệm A: số mẫu huấn luyện

Thiết kế. A ∈ {200, 400, …, 2000}, mỗi mức lấy A mẫu đầu của train pool, mạng [10], các siêu tham số khác như baseline, 3 seed (30 run).

Bảng 4.5. *Kết quả Thực nghiệm A (* hậu kiểm). Nguồn results/expA/summary.csv, posthoc_test.log.*

A	Train MSE	Val MSE	Test MSE*	Val MAE	Test MAE*	Test RMSE*
200	5.43e-5 ± 1.4e-5	7.06e-5 ± 1.8e-5	9.24e-5 ± 3.0e-5	6.26e-3 ± 8.9e-4	6.92e-3 ± 1.1e-3	9.52e-3 ± 1.6e-3
400	3.45e-5 ± 1.3e-5	3.93e-5 ± 1.3e-5	4.07e-5 ± 1.6e-5	4.73e-3 ± 8.3e-4	4.97e-3 ± 8.4e-4	6.31e-3 ± 1.2e-3
600	1.95e-5 ± 8.3e-6	2.16e-5 ± 7.8e-6	2.36e-5 ± 1.1e-5	3.56e-3 ± 6.2e-4	3.61e-3 ± 8.8e-4	4.77e-3 ± 1.1e-3
800	2.74e-5 ± 1.7e-5	2.73e-5 ± 1.7e-5	3.19e-5 ± 2.1e-5	4.05e-3 ± 1.4e-3	4.36e-3 ± 1.5e-3	5.46e-3 ± 1.8e-3
1000	2.21e-5 ± 6.1e-6	1.89e-5 ± 5.9e-6	2.28e-5 ± 1.4e-5	3.35e-3 ± 5.6e-4	3.53e-3 ± 1.1e-3	4.63e-3 ± 1.4e-3
1200	2.19e-5 ± 9.4e-6	2.23e-5 ± 9.3e-6	2.33e-5 ± 9.4e-6	3.56e-3 ± 6.5e-4	3.72e-3 ± 6.7e-4	4.76e-3 ± 9.3e-4
1400	1.80e-5 ± 7.1e-6	1.62e-5 ± 4.7e-6	2.20e-5 ± 9.3e-6	3.13e-3 ± 3.3e-4	3.52e-3 ± 7.8e-4	4.62e-3 ± 9.5e-4
1600	1.72e-5 ± 4.5e-6	1.52e-5 ± 1.4e-6	1.39e-5 ± 2.6e-6	3.06e-3 ± 9.1e-5	2.94e-3 ± 2.4e-4	3.72e-3 ± 3.4e-4
1800	2.01e-5 ± 2.6e-6	1.75e-5 ± 3.5e-6	2.13e-5 ± 2.3e-6	3.20e-3 ± 3.3e-4	3.48e-3 ± 1.7e-4	4.62e-3 ± 2.5e-4
2000	3.24e-5 ± 1.2e-5	2.57e-5 ± 8.4e-6	3.35e-5 ± 1.9e-5	3.91e-3 ± 5.9e-4	4.37e-3 ± 1.2e-3	5.64e-3 ± 1.6e-3



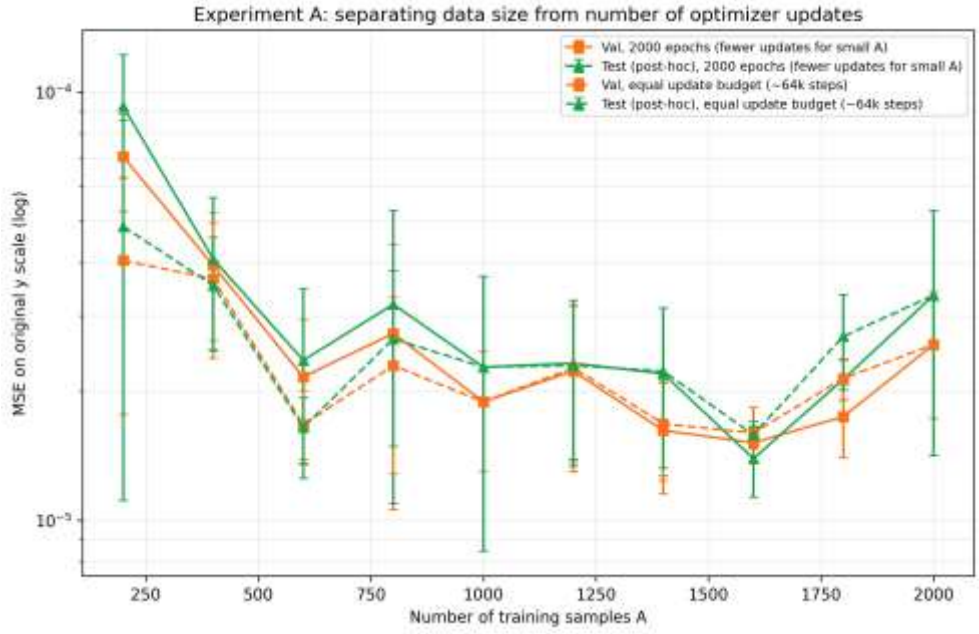
Hình 4.3. *Train, validation và test (hậu kiểm) MSE theo A, error bar ± std qua 3 seed; đường ngang là mô hình hằng số.*

Nhận xét: Val MSE giảm từ 7.06e-5 (A = 200) xuống 2.16e-5 (A = 600) rồi dao động trong khoảng 1.5e-5–2.7e-5; thấp nhất ở A = 1600 (1.52e-5), tăng lại ở A = 2000 (2.57e-5). Train và validation gần nhau ở mọi mức, không thấy quá khớp tăng theo A. Test (hậu kiểm) đi cùng xu hướng với validation; khoảng ± std của test và validation chồng nhau ở mọi mức A, nên không thấy validation lệch hẳn so với test (3 seed, test 100 mẫu).

Đối chứng cùng ngân sách bước cập nhật. Log cho thấy batch cố định 32 làm A nhỏ có ít bước cập nhật hơn: A = 200 chỉ 7 bước/epoch, tức khoảng 14 000 bước trong 2000 epoch, so với 64 000 bước của A = 1000; 8/9 run ở A = 200–600 chạm trần epoch khi vẫn đang cải thiện. Nhóm chạy lại A với max_epochs và patience nhân 32/[A/32] để mọi mức A có cùng khoảng 64 000 bước (configs/expA_steps.json). Kết quả (Bảng 4.6, Hình 4.4): A = 200 giảm từ 7.06e-5 xuống 4.04e-5 (−43%), các mức A ≤ 800 giảm 7–43%; theo cặp cùng seed, 10/12 cặp run ở A ≤ 800 giảm val MSE (từ 4.5% đến 72%) và 2 cặp gần như không đổi (chênh < 0.01%). Như vậy một phần đáng kể của "lỗi cao khi ít dữ liệu" là do huấn luyện chưa đủ. Sau khi cân bằng số bước, A = 200 vẫn kém A = 1000 khoảng 2 lần (4.04e-5 ± 2.3e-5 so với 1.89e-5 ± 5.9e-6); khoảng ± std chồng nhau với 3 seed nên đây là dấu hiệu chứ chưa phải kết luận chắc. Từ A ≥ 600, val MSE nằm trong dải 1.6e-5–2.6e-5 với ± std chồng nhau: giới hạn chính là dung lượng của mạng [10] (41 tham số), không phải dữ liệu.

Bảng 4.6. *Đối chứng A cùng ngân sách khoảng 64 000 bước cập nhật (* hậu kiểm). Nguồn results/expA_steps/summary.csv.*

A	Số epoch chạy	Val MSE	Test MSE*
200	6551 ± 2514	4.04e-5 ± 2.3e-5	4.85e-5 ± 3.7e-5
400	3477 ± 984	3.66e-5 ± 1.3e-5	3.53e-5 ± 1.0e-5
600	2737 ± 1093	1.67e-5 ± 3.2e-6	1.65e-5 ± 2.9e-6
800	2272 ± 406	2.30e-5 ± 1.0e-5	2.65e-5 ± 1.2e-5
1000	1792 ± 346	1.89e-5 ± 5.9e-6	2.28e-5 ± 1.4e-5
1200	1259 ± 385	2.27e-5 ± 9.0e-6	2.30e-5 ± 9.6e-6
1400	1138 ± 365	1.68e-5 ± 4.4e-6	2.23e-5 ± 9.0e-6
1600	896 ± 366	1.60e-5 ± 2.3e-6	1.58e-5 ± 1.1e-6
1800	731 ± 161	2.14e-5 ± 2.3e-6	2.69e-5 ± 6.7e-6
2000	581 ± 168	2.57e-5 ± 8.4e-6	3.35e-5 ± 1.9e-5



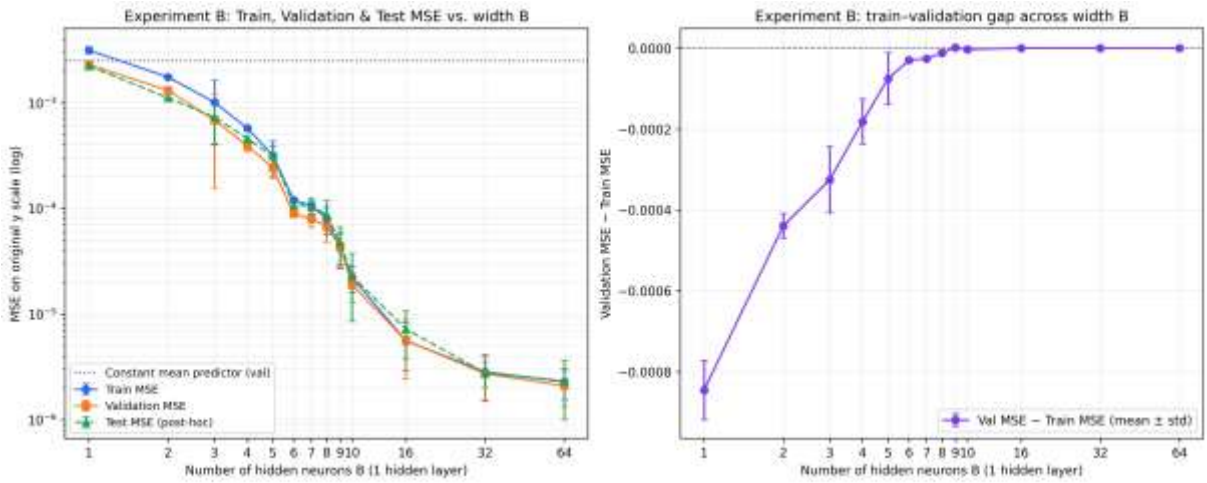
Hình 4.4. *Val và test MSE theo A: liền nét là Thực nghiệm A (2000 epoch), đứt nét là đối chứng cùng ngân sách bước.*

4.3. Thực nghiệm B: số nơ-ron của tầng ẩn

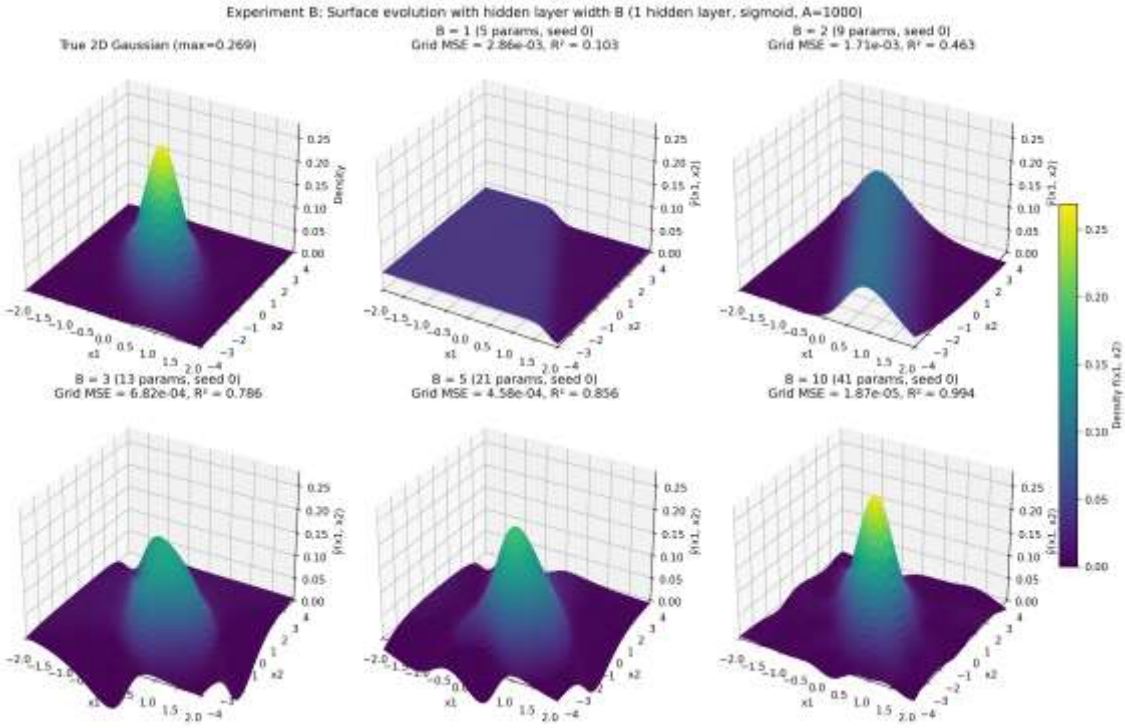
Thiết kế. Mạng một tầng ẩn [2, B, 1], B = 1…10 ở A = 1000 (30 run). Để xem xu hướng khi rộng hơn, nhóm chạy thêm B = 16, 32, 64 (expB_wide) và dùng B = 38, 65, 93 từ đối chứng của Thực nghiệm C. Để tìm quá khớp, toàn bộ dải B được lặp lại ở A = 200 (expB_A200, 39 run).

Bảng 4.7. *Thực nghiệm B tại A = 1000 (B = 16–64 mở rộng; B = 38, 65, 93 lấy từ đối chứng C; * hậu kiểm).*

B	Tham số	Train MSE	Val MSE	Test MSE*	Val MAE	Epoch tốt nhất
1	5	3.14e-3 ± 2.2e-4	2.29e-3 ± 1.5e-4	2.23e-3 ± 1.1e-4	3.24e-2 ± 2.2e-3	515 ± 353
2	9	1.75e-3 ± 3.2e-5	1.31e-3 ± 1.8e-5	1.11e-3 ± 1.2e-5	2.20e-2 ± 3.9e-4	69 ± 52
3	13	1.01e-3 ± 6.1e-4	6.84e-4 ± 5.3e-4	7.28e-4 ± 3.2e-4	1.58e-2 ± 5.2e-3	334 ± 244
4	17	5.73e-4 ± 2.9e-5	3.91e-4 ± 4.5e-5	4.57e-4 ± 1.9e-5	1.38e-2 ± 6.4e-4	637 ± 583
5	21	3.18e-4 ± 1.1e-4	2.43e-4 ± 5.2e-5	3.12e-4 ± 7.3e-5	1.09e-2 ± 1.4e-3	487 ± 189
6	25	1.19e-4 ± 3.5e-7	8.91e-5 ± 2.2e-6	1.07e-4 ± 1.8e-5	6.88e-3 ± 3.1e-4	1046 ± 461
7	29	1.06e-4 ± 1.1e-5	7.98e-5 ± 1.4e-5	1.06e-4 ± 1.8e-5	6.56e-3 ± 7.7e-4	1069 ± 854
8	33	7.95e-5 ± 2.3e-5	6.73e-5 ± 1.9e-5	8.74e-5 ± 3.0e-5	6.13e-3 ± 7.8e-4	913 ± 594
9	37	4.21e-5 ± 1.5e-5	4.34e-5 ± 1.5e-5	4.76e-5 ± 1.8e-5	4.98e-3 ± 8.2e-4	1618 ± 224
10	41	2.21e-5 ± 6.1e-6	1.89e-5 ± 5.9e-6	2.28e-5 ± 1.4e-5	3.35e-3 ± 5.6e-4	1653 ± 411
16	65	5.58e-6 ± 2.7e-6	5.63e-6 ± 3.2e-6	7.19e-6 ± 3.5e-6	1.74e-3 ± 4.0e-4	1921 ± 55
32	129	2.82e-6 ± 1.3e-6	2.72e-6 ± 1.2e-6	2.76e-6 ± 7.6e-7	1.32e-3 ± 2.7e-4	1277 ± 531
38	153	2.38e-6 ± 1.1e-6	2.42e-6 ± 1.6e-6	2.96e-6 ± 1.8e-6	1.18e-3 ± 3.8e-4	1378 ± 582
64	257	2.28e-6 ± 7.4e-7	2.09e-6 ± 7.6e-7	2.32e-6 ± 1.3e-6	1.10e-3 ± 2.0e-4	1355 ± 133
65	261	1.87e-6 ± 9.5e-7	1.68e-6 ± 8.7e-7	1.92e-6 ± 1.2e-6	9.95e-4 ± 2.6e-4	1084 ± 221
93	373	1.11e-6 ± 1.6e-7	1.46e-6 ± 4.3e-7	1.41e-6 ± 2.0e-7	9.06e-4 ± 9.8e-5	1266 ± 445



Hình 4.5. *Trái: MSE theo B tại A = 1000 (trục B theo $\log_2$). Phải: khoảng cách val − train MSE.*



Hình 4.6. *Bề mặt thật và bề mặt xấp xỉ của mạng một tầng với B = 1, 2, 3, 5, 10 (seed 0).*

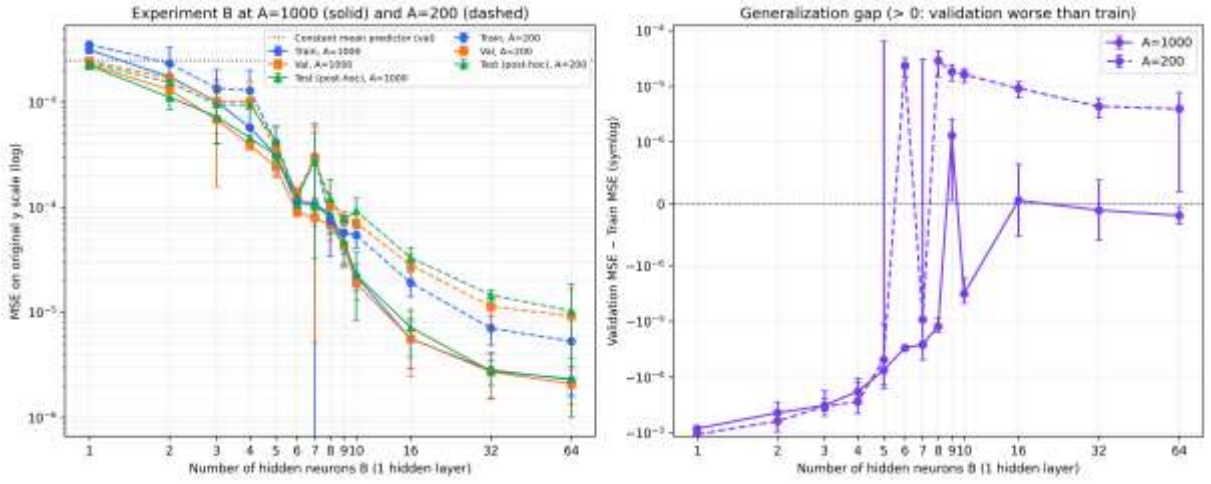
Thiếu khớp khi B nhỏ. Val MSE giảm đều hơn 120 lần từ B = 1 (2.29e-3, gần mức hằng số 2.49e-3) đến B = 10 (1.89e-5). Hình 4.6 giải thích bằng hình học: với B = 1, đầu ra $w \cdot \sigma(u_1x_1 + u_2x_2 + b) + c$ là một "sườn dốc" chỉ thay đổi theo một hướng, không thể tạo đỉnh; B = 2 tạo được một dải chạy dọc trục chính của elip nhưng chưa khép kín hai đầu; từ B ≈ 5 trở lên, nhiều sườn dốc xoay theo các hướng khác nhau mới ghép được thành đỉnh elip nghiêng.

Không có quá khớp ở A = 1000. Val MSE tiếp tục giảm khi mở rộng: 5.63e-6 (B = 16), 2.09e-6 (B = 64), 1.46e-6 (B = 93); khoảng cách val − train tiến về 0. Ở B nhỏ, val thấp hơn train vì nhãn validation có phương sai thấp hơn (2.47e-3 so với 3.38e-3 của 1000 mẫu train đầu tiên, tỉ số 0.73), xấp xỉ tỉ số Val/Train MSE ở B = 1 (2.29e-3 /

3.14e-3 = 0.73); tập validation cũng có tỉ lệ điểm y ≈ 0 cao hơn (66% so với 62.6%), nên mô hình thiếu khớp bị phạt ít hơn trên validation.

Bảng 4.8. *Thực nghiệm B lặp lại tại A = 200 (* hậu kiểm). Nguồn results/expB_A200/summary.csv.*

B	Tham số	Train MSE	Val MSE	Test MSE*	Val/Train
1	5	3.54e-3 ± 2.3e-5	2.47e-3 ± 5.5e-6	2.36e-3 ± 4.8e-6	0.70
2	9	2.33e-3 ± 1.0e-3	1.71e-3 ± 6.6e-4	1.55e-3 ± 7.0e-4	0.73
3	13	1.35e-3 ± 6.7e-4	1.02e-3 ± 5.1e-4	9.56e-4 ± 2.9e-4	0.75
4	17	1.29e-3 ± 7.1e-4	1.01e-3 ± 5.4e-4	9.37e-4 ± 3.6e-4	0.78
5	21	4.13e-4 ± 1.8e-4	3.64e-4 ± 6.7e-5	4.27e-4 ± 1.6e-4	0.88
6	25	1.14e-4 ± 8.4e-6	1.37e-4 ± 1.7e-5	1.26e-4 ± 1.6e-5	1.20
7	29	3.04e-4 ± 3.2e-4	2.94e-4 ± 2.9e-4	2.73e-4 ± 2.4e-4	0.97
8	33	7.37e-5 ± 3.9e-5	1.03e-4 ± 3.0e-5	1.20e-4 ± 6.4e-5	1.39
9	37	5.74e-5 ± 2.1e-5	7.57e-5 ± 1.6e-5	7.79e-5 ± 6.2e-6	1.32
10	41	5.43e-5 ± 1.4e-5	7.06e-5 ± 1.8e-5	9.24e-5 ± 3.0e-5	1.30
16	65	1.91e-5 ± 5.1e-6	2.82e-5 ± 7.3e-6	3.31e-5 ± 7.9e-6	1.48
32	129	7.05e-6 ± 2.2e-6	1.14e-5 ± 3.7e-6	1.46e-5 ± 1.8e-6	1.61
64	257	5.30e-6 ± 3.6e-6	9.21e-6 ± 7.4e-6	1.03e-5 ± 8.3e-6	1.74



Hình 4.7. *Thực nghiệm B tại A = 1000 (liền nét) và A = 200 (đứt nét). Phải: khoảng cách val − train (thang symlog).*

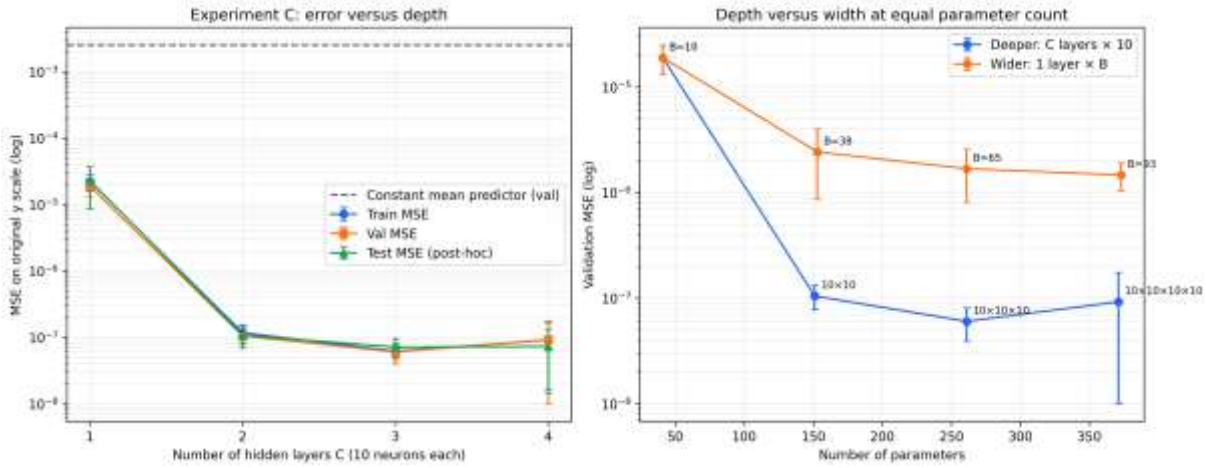
Ít dữ liệu: có khoảng cách tổng quát hóa nhưng chưa có chữ U. Ở A = 200, từ B ≥ 8 val MSE luôn lớn hơn train; từ B = 10 trở đi tỉ số Val/Train tăng theo B, từ 1.30 (B = 10) lên 1.74 (B = 64), trong khi ở A = 1000 tỉ số này xấp xỉ 1 (Bảng 4.8, Hình 4.7). Đây là dấu hiệu sớm của quá khớp (ngoại lệ B = 7: seed 0 hội tụ kém, train MSE 6.69e-4 và val 6.25e-4, nên kéo trung bình lên; đó là lỗi tối ưu chứ không phải quá khớp). Tuy vậy val và test vẫn giảm theo B (val từ 7.06e-5 xuống 9.21e-6), tức chưa có đường chữ U. 17/39 run chạm trần 2000 epoch, nên khoảng cách có thể rõ hơn nếu huấn luyện lâu hơn.

4.4. Thực nghiệm C: số tầng ẩn

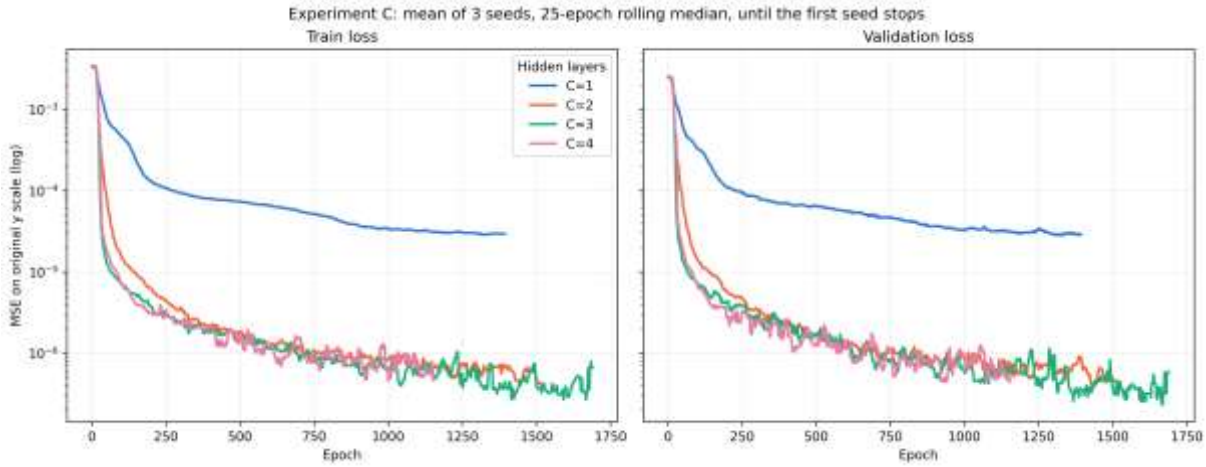
Thiết kế. C ∈ {1, 2, 3, 4} tầng ẩn, mỗi tầng 10 nơ-ron, các yếu tố khác như baseline (12 run). Tăng C cũng tăng số tham số (41 → 371), nên nhóm thêm đối chứng cùng số tham số: mạng một tầng B = 38, 65, 93 (153, 261, 373 tham số, theo công thức 4B + 1). Thời gian đo bằng time.perf_counter() quanh fit(), cùng máy, 1 luồng, các run chạy tuần tự. Mỗi epoch ghi chuẩn gradient từng lớp.

Bảng 4.9. *Thực nghiệm C và đối chứng mạng một tầng cùng số tham số (* hậu kiểm). Nguồn results/expC/, results/expC_ctrl/.*

Mạng	Tham số	Train MSE	Val MSE	Test MSE*	Val MAE	Epoch tốt nhất	Thời gian (s)
[10]	41	2.21e-5 ± 6.1e-6	1.89e-5 ± 5.9e-6	2.28e-5 ± 1.4e-5	3.35e-3 ± 5.6e-4	1653 ± 411	36.3 ± 7.2
[10,10]	151	1.17e-7 ± 3.4e-8	1.05e-7 ± 2.7e-8	1.07e-7 ± 3.9e-8	2.35e-4 ± 3.0e-5	1653 ± 315	41.8 ± 4.7
[10,10,10]	261	6.18e-8 ± 1.2e-8	6.02e-8 ± 2.1e-8	7.13e-8 ± 2.4e-8	1.80e-4 ± 2.6e-5	1667 ± 208	47.2 ± 4.2
[10,10,10,10]	371	9.09e-8 ± 7.5e-8	9.21e-8 ± 8.2e-8	7.30e-8 ± 5.9e-8	2.25e-4 ± 1.1e-4	1298 ± 442	45.5 ± 12.9
[38]	153	2.38e-6 ± 1.1e-6	2.42e-6 ± 1.6e-6	2.96e-6 ± 1.8e-6	1.18e-3 ± 3.8e-4	1378 ± 582	32.3 ± 13.2
[65]	261	1.87e-6 ± 9.5e-7	1.68e-6 ± 8.7e-7	1.92e-6 ± 1.2e-6	9.95e-4 ± 2.6e-4	1084 ± 221	26.0 ± 5.2
[93]	373	1.11e-6 ± 1.6e-7	1.46e-6 ± 4.3e-7	1.41e-6 ± 2.0e-7	9.06e-4 ± 9.8e-5	1266 ± 445	29.6 ± 9.1



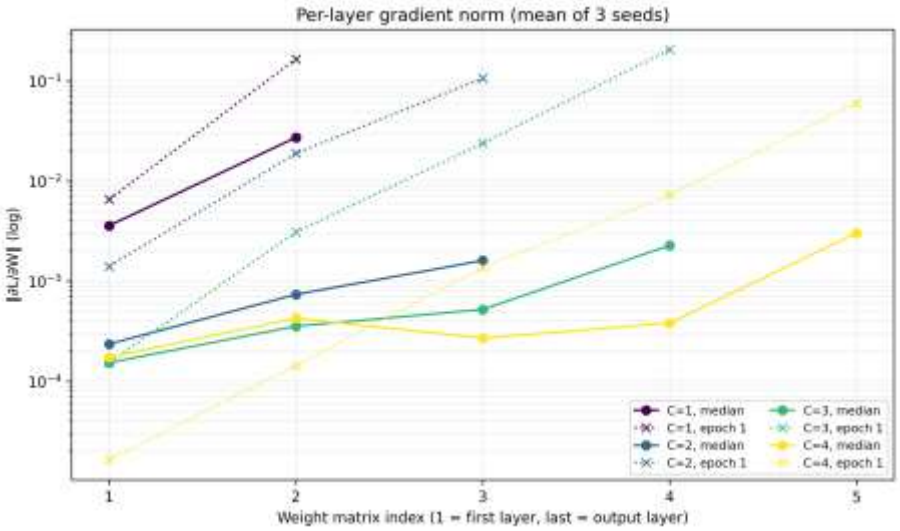
Hình 4.8. *Trái: MSE theo C. Phải: val MSE của mạng sâu và mạng một tầng rộng theo số tham số.*



Hình 4.9. *Train loss và val loss theo epoch với C = 1…4 (trung bình 3 seed, trung vị trượt 25 epoch). Đường cong dừng ở epoch dừng sớm nhất trong 3 seed.*

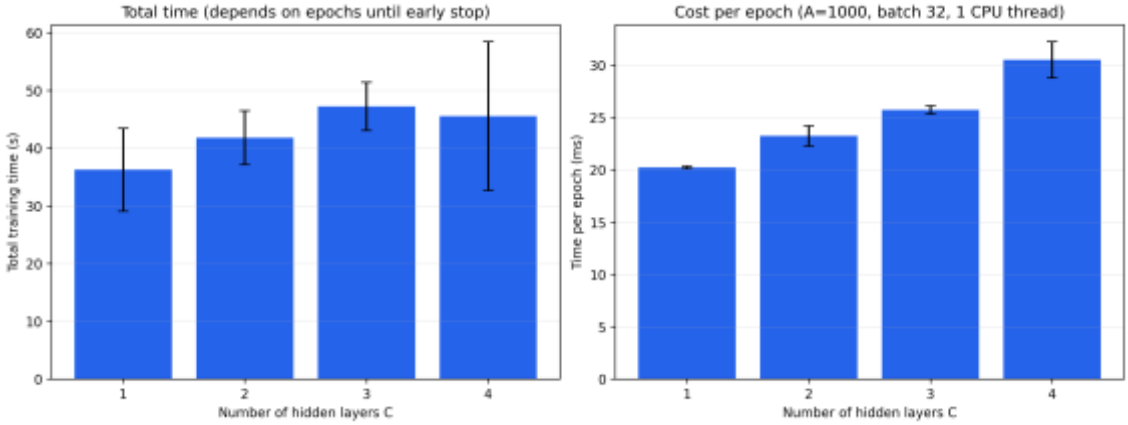
Thêm tầng làm lỗi giảm mạnh, nhưng chỉ đến C = 2–3. Val MSE giảm khoảng 180 lần từ C = 1 (1.89e-5) sang C = 2 (1.05e-7). Từ C = 2 đến C = 4 lỗi cùng bậc 1e-7, khoảng ± std của C = 3 và C = 4 chồng nhau. Với C ≥ 2, loss xuống dưới 1e-5 trong khoảng 200 epoch đầu, còn C = 1 giảm chậm về quanh 3e-5 (Hình 4.9). Train, validation và test gần bằng nhau ở mọi C, không có quá khớp.

Lợi ích đến từ độ sâu, không chỉ từ số tham số. Ở cùng số tham số, mạng sâu có val MSE thấp hơn mạng một tầng 16–28 lần (151 so với 153 tham số: 1.05e-7 so với 2.42e-6; 261: 6.02e-8 so với 1.68e-6; 371/373: 9.21e-8 so với 1.46e-6). Mạng rộng giảm chậm hơn nhiều: val MSE chỉ từ 2.42e-6 (B = 38) xuống 1.46e-6 (B = 93), với độ lệch chuẩn giữa seed bằng 29–66% giá trị trung bình.



Hình 4.10. *Chuẩn gradient từng lớp ở epoch đầu và trung vị qua các epoch (trung bình 3 seed).*

Dấu hiệu vanishing gradient xuất hiện ở đầu huấn luyện nhưng không cản trở huấn luyện. Ở epoch 1 (đo ở batch cuối của epoch), chuẩn gradient giảm dần khi lan ngược qua từng tầng, mỗi tầng khoảng 5–20 lần (Hình 4.10): với C = 4, lớp đầu chỉ 1.7e-5 so với 6.0e-2 ở lớp ra, chênh khoảng 3600 lần; với C = 1 chênh khoảng 25 lần. Trong quá trình huấn luyện, trung vị chuẩn gradient của các tầng ẩn chỉ còn chênh dưới 3.5 lần, và mạng sâu vẫn đạt lỗi thấp nhất.



Hình 4.11. *Tổng thời gian huấn luyện (trái) và thời gian mỗi epoch (phải) theo C.*

Thời gian. Thời gian mỗi epoch tăng từ 20.2 ms (C = 1) lên 30.5 ms (C = 4), khoảng +3.4 ms mỗi tầng, trong khi mạng một tầng 93 nơ-ron (373 tham số) vẫn chỉ 20.2 ms (Hình 4.11). Với mạng nhỏ, chi phí bị chi phối bởi số phép tính tuần tự theo tầng, không bởi kích thước ma trận. Tổng thời gian (khoảng 36–47 s) dao động nhiều hơn vì phụ thuộc số epoch đến khi dừng sớm.

4.5. Mô hình cuối và đánh giá trên tập test

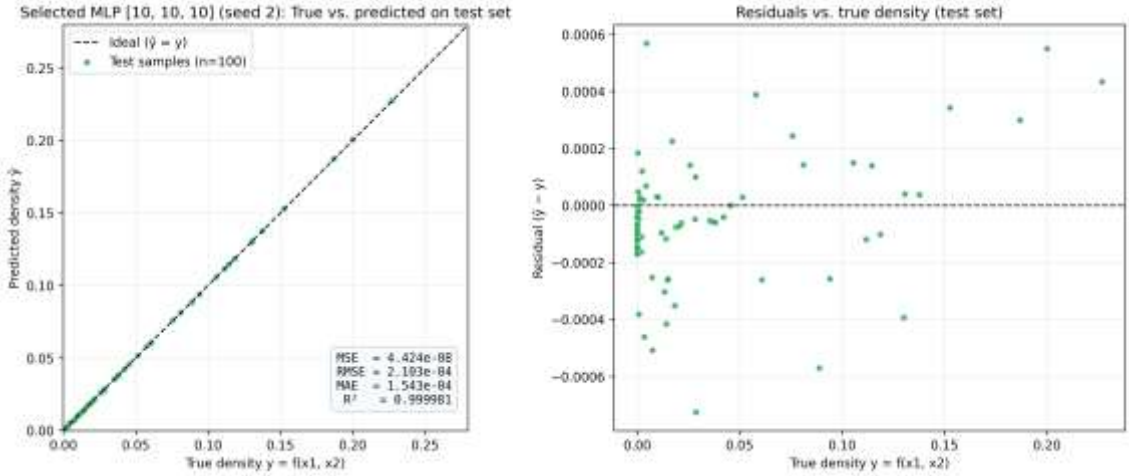
Chọn mô hình. results/summary_all.csv gộp 53 dòng kết quả của 6 nhóm (A, A_steps, B, B_A200, C, C_ctrl; không gồm các lưới grid_*). Cấu hình có val MSE trung bình thấp nhất là C = 3 [10,10,10] (6.02e-8), tiếp theo C = 4 (9.21e-8) và C = 2 (1.05e-7).

Theo trung vị và giá trị nhỏ nhất của 3 seed thì C = 4 nhỉnh hơn (5.34e-8 so với 5.68e-8; 3.65e-8 so với 4.14e-8), nên với 3 seed hai cấu hình không phân biệt được. Theo quy tắc trung bình val thấp nhất, nhóm chọn C = 3 (261 tham số, sigmoid, Adam lr 0.01, A = 1000), đóng băng vào configs/final.json, rồi mới chạy --eval-test một lần. Checkpoint results/final/best.pt lấy từ seed có val thấp nhất (seed 2, epoch 1490). Vì tập test chỉ có 100 điểm và 61 điểm có y < 0.01, nhóm đánh giá thêm trên lưới đều 100 × 100 phủ toàn miền.

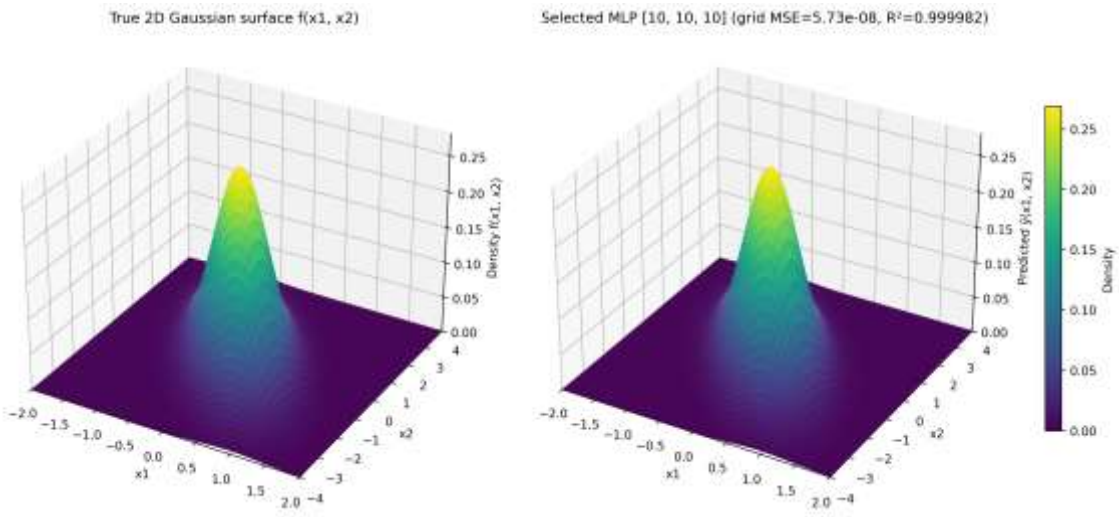
Bảng 4.10. *Đánh giá cuối trên test (N = 100) và lưới 100 × 100 (mean ± std qua 3 seed). Nguồn results/final/aggregate.csv.*

Mô hình	Tham số	Val MSE	Test MSE	Test MAE	Test RMSE	Test R^2	Grid MSE
Hằng số (trung bình train)	1	2.49e-3	2.37e-3	3.64e-2	4.87e-2	-0.00734	—
Baseline [10]	41	1.89e-5 ± 5.9e-6	2.28e-5 ± 1.4e-5	3.53e-3 ± 1.1e-3	4.63e-3 ± 1.4e-3	0.99030 ± 0.00609	2.53e-5 ± 7.1e-6
Một tầng [93]	373	1.46e-6 ± 4.3e-7	1.41e-6 ± 2.0e-7	8.83e-4 ± 2.0e-5	1.18e-3 ± 8.7e-5	0.99940 ± 0.00009	1.53e-6 ± 3.3e-7
Mô hình chọn [10,10,10]	261	6.02e-8 ± 2.1e-8	7.13e-8 ± 2.4e-8	1.94e-4 ± 3.4e-5	2.64e-4 ± 4.7e-5	0.999970 ± 0.000010	7.34e-8 ± 1.4e-8

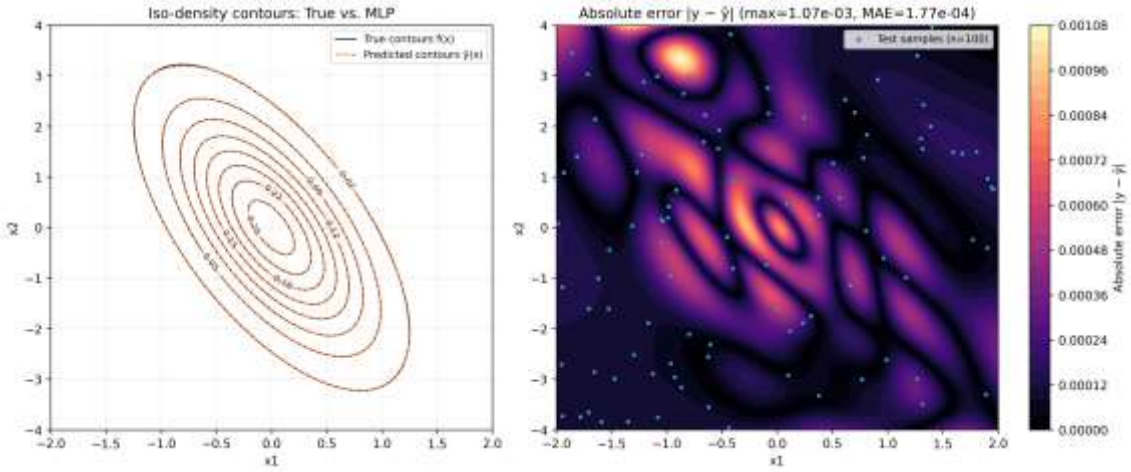
Checkpoint seed 2 (best.pt) đạt Test MSE 4.42e-8, Test MAE 1.54e-4, R^2 = 0.99998, Grid MSE 5.73e-8 và sai số tuyệt đối lớn nhất trên lưới 1.07e-3, dưới 0.4% chiều cao đỉnh (results/final/metrics.json).



Hình 4.12. *Trái: y thật và ŷ dự đoán trên 100 mẫu test cùng đường ŷ = y. Phải: phần dư theo y. Checkpoint seed 2.*



Hình 4.13. *Bề mặt Gaussian thật (trái) và bề mặt xấp xỉ của MLP [10, 10, 10] (phải), cùng thang z. Checkpoint seed 2.*



Hình 4.14. *Trái: đường đồng mức của hàm thật (liền) và hàm xấp xỉ (đứt). Phải: bản đồ |y − ŷ| trên lưới kèm vị trí 100 mẫu test. Checkpoint seed 2.*

Nhận xét: Test MSE (7.13e-8) và Grid MSE (7.34e-8) rất sát Val MSE (6.02e-8), nên việc chọn theo validation không làm mô hình khớp riêng tập validation. Các điểm test nằm sát đường ŷ = y trên toàn dải y của tập test (0–0.227; Hình 4.12), còn vùng y > 0.227 chỉ được kiểm tra qua lưới; đường đồng mức của mô hình chồng khít hàm thật (Hình 4.14). Với checkpoint seed 2, sai số tuyệt đối lớn nhất trên lưới (1.07e-3, ŷ thấp hơn y) nằm ở ($x_1 \approx -0.87$; $x_2 \approx 3.35$), đầu trên của trục dài, nơi y ≈ 0.016. Sai số trung bình ở vùng y > 0.2 là 4.5e-4 (tối đa 9.4e-4), so với 1.4e-4 ở vùng y < 0.02.

Chương 5. Thảo luận

5.1. Vì sao không thấy đường chữ U của quá khớp

Hình minh họa trong đề và nhiều giáo trình cho thấy lỗi validation giảm rồi tăng lại khi mô hình quá lớn. Nhóm không quan sát được điều này, kể cả khi giảm còn A = 200 và tăng B tới 64. Có ba nguyên nhân có thể (chưa được kiểm chứng bằng thí nghiệm riêng, xem mục 5.5):

1. Nhãn không có nhiễu. Quá khớp nặng xảy ra khi mô hình dùng dung lượng thừa để ghi nhớ nhiễu ở các điểm train. Ở đây y = f(x) chính xác, nên khớp đúng điểm train cũng là khớp đúng hàm.
2. Hàm mục tiêu trơn và dữ liệu phủ đủ dày. Một hàm Gaussian 2D trơn, không có chi tiết nhỏ; 200–2000 điểm đều trên miền 2D đủ để mạng nội suy tốt giữa các điểm.
3. Early stopping theo validation. Checkpoint luôn lấy ở epoch có val loss thấp nhất, nên mô hình không kịp đi vào vùng quá khớp.

Điều nhóm thấy được là dấu hiệu sớm: ở A = 200, tỉ số Val/Train tăng từ 1.30 lên 1.74 khi B tăng từ 10 lên 64. Muốn thấy rõ chữ U, cần thêm nhiễu vào nhãn hoặc giảm A mạnh hơn nữa (mục 5.5).

5.2. Vì sao mạng sâu tốt hơn mạng rộng

Mỗi nơ-ron sigmoid ở tầng đầu chỉ tạo một "sườn dốc" theo một hướng trong mặt phẳng (x_1, x_2). Mạng một tầng phải cộng tuyến tính rất nhiều sườn dốc mới tạo được một đỉnh khép kín hình elip, nên lỗi giảm chậm: ở B = 93 (373 tham số) val MSE vẫn cao hơn mạng sâu cùng số tham số khoảng 16 lần (1.46e-6 so với 9.21e-8). Tầng thứ hai lấy các sườn dốc làm đầu vào và kết hợp phi tuyến chúng thành vùng lồi bao quanh đỉnh, nên biểu diễn hình dạng này hiệu quả hơn nhiều với cùng số tham số. Định lý xấp xỉ phổ quát [5], [6] chỉ bảo đảm mạng một tầng *có thể* xấp xỉ, không nói nó cần bao nhiêu nơ-ron; số liệu cho thấy với hàm này, cái giá đó cao hơn hẳn so với thêm tầng. Đây là giải thích giả thuyết; Hình 4.6 chỉ minh họa cho mạng một tầng.

5.3. Những kết quả không như kỳ vọng

- Ngược với Sample_Report. Báo cáo mẫu kết luận tăng tầng làm lỗi tăng do vanishing gradient. Nhóm đo được dấu hiệu vanishing gradient ở đầu huấn luyện (batch cuối của epoch 1: lớp đầu nhỏ hơn lớp ra khoảng 3600 lần với C = 4), nhưng nó không làm hỏng huấn luyện. Giả thuyết hợp lý là Adam chia bước cập nhật cho $\sqrt{\hat{v}_t}$ nên tham số có gradient nhỏ vẫn được cập nhật với bước cỡ lr; nhóm chưa chạy SGD thuần trên mạng sâu nên chưa kiểm chứng được giả thuyết này.
- Yếu tố lẫn giữa số mẫu và số bước ở A. Kết quả gốc của Thực nghiệm A phóng đại ảnh hưởng của số mẫu, vì A nhỏ được huấn luyện ít bước hơn. Thí nghiệm đối chứng cho thấy val MSE ở A = 200 giảm 43% (7.06e-5 → 4.04e-5) khi cấp đủ bước.
- A = 2000 có lỗi tăng nhẹ. Val MSE ở A = 2000 (2.57e-5) cao hơn A = 1600 (1.52e-5). Đây không phải do ngân sách bước: ở đối chứng, A = 2000 cho kết quả trùng khớp vì cả 3 seed đều dừng sớm trước giới hạn mới. Cả 3 seed ở A = 2000 (1.87e-5, 3.51e-5, 2.33e-5) đều cao hơn cả 3 seed ở A = 1600 (1.42e-5–1.68e-5) và dừng sớm ở epoch 553–870; với 3 seed chưa đủ để kết luận đó là xu hướng.

- Validation thấp hơn train khi thiếu khớp. Ở B nhỏ, val MSE thấp hơn train MSE; nguyên nhân là nhãn validation có phương sai thấp hơn (2.47e-3 so với 3.38e-3, tỉ số 0.73, xấp xỉ tỉ số Val/Train MSE ở B = 1), không phải mô hình tổng quát hóa tốt hơn.

5.4. Hạn chế

- Trần 2000 epoch. Tùy thí nghiệm, 6.7–50% run có epoch tốt nhất nằm trong 200 epoch cuối, tức có thể vẫn đang cải thiện khi bị cắt (11/30 run A, 17/39 run B ở A = 200). Lỗi của một số cấu hình có thể bị đánh giá cao hơn thực tế.
- Chỉ 3 seed. Nhiều chênh lệch (batch size, $A \geq 600$, C = 3 so với C = 4, Adam so với Nadam, RMSProp) nhỏ hơn độ lệch chuẩn nên không kết luận được.
- Tìm siêu tham số theo từng tọa độ. Nhóm cố định lần lượt activation + lr, rồi optimizer, batch, patience thay vì thử mọi tổ hợp; Adam, batch 32 và patience 200 là chọn trước rồi mới kiểm chứng. Có thể bỏ sót tổ hợp tốt hơn (ví dụ tanh với optimizer khác), và kết luận về optimizer chỉ làm trên mạng một tầng.
- Phạm vi C. Chỉ xét tối đa 4 tầng, sigmoid và Adam; với mạng sâu hơn hoặc SGD, vanishing gradient có thể ảnh hưởng rõ hơn.
- Đánh giá thời gian. Chỉ chạy CPU một luồng. Thời gian của 90 run bổ sung (expA_steps, expB_wide, expB_A200, grid_batch) đo khi 4 tiến trình chạy song song, nên không so sánh được với các run khác; báo cáo chỉ dùng thời gian của Thực nghiệm C.
- Lấy mẫu đều trên miền khác hình scatter minh họa trong đề (điểm dồn quanh tâm), làm 62.6% nhãn gần 0; nhóm bù lại bằng R^2, đánh giá trên lưới và bản đồ sai số.

5.5. Đề xuất cải tiến

- Thêm nhiễu Gaussian vào nhãn hoặc giảm A xuống 50–100 để kiểm tra có xuất hiện đường chữ U hay không.
- Tăng trần epoch, hoặc cố định tổng số bước cập nhật cho mọi thí nghiệm, để loại hẳn ảnh hưởng của việc huấn luyện chưa đủ.
- Chạy 5–10 seed cho các cấu hình gần nhau để tách được C = 3 với C = 4 và các optimizer.
- So sánh Adam với SGD trên mạng sâu C = 4–8 để kiểm chứng giả thuyết Adam bù vanishing gradient.
- Giảm learning rate theo lịch (learning-rate schedule) để làm mượt đường val loss của mạng sâu, vốn dao động mạnh ở lr 0.01 khi lỗi đã rất nhỏ.

Chương 6. Kết luận

Nhóm đã cài đặt MLP tùy biến bằng PyTorch với vòng lặp train/validation tự viết, early stopping và checkpoint theo validation, và chạy hơn 320 run có ghi log để khảo sát ba yếu tố A, B, C. Kết luận chính:

1. Số mẫu (A): lỗi giảm mạnh đến A ≈ 600 rồi chững; một phần lỗi ở A nhỏ là do ít bước cập nhật. Với mạng [10], giới hạn chính là dung lượng mô hình chứ không phải dữ liệu.
2. Độ rộng (B): B nhỏ thiếu khớp rõ, lỗi giảm hơn 120 lần từ B = 1 đến B = 10 và tiếp tục giảm tới B = 93. Không có quá khớp ở A = 1000; ở A = 200 xuất hiện khoảng cách train–validation nhưng chưa có đường chữ U, do nhãn không nhiễu và có early stopping.
3. Độ sâu (C): thêm tầng giảm lỗi khoảng 180 lần từ C = 1 sang C = 2, và mạng sâu tốt hơn mạng rộng cùng số tham số 16–28 lần. Dấu hiệu vanishing gradient xuất hiện ở đầu huấn luyện (epoch 1) nhưng không cản trở huấn luyện với Adam.
4. Mô hình cuối [10,10,10], chọn hoàn toàn bằng validation, đạt Test MSE 7.13e-8 ± 2.36e-8, R^2 = 0.99997 và Grid MSE 7.34e-8.

Mọi lựa chọn được ghi trong nhật ký quyết định (Phụ lục B) cùng file minh chứng; mọi bảng số liệu sinh tự động từ log bằng src/make_tables.py.

Tài liệu tham khảo

1. PyTorch Documentation, *torch.nn*, *torch.optim*, *torch.utils.data*. https://pytorch.org/docs/stable/
2. I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
3. D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.
4. X. Glorot, Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," *AISTATS*, 2010.
5. G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, 2(4), 1989.
6. K. Hornik, M. Stinchcombe, H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, 2(5), 1989.
7. Đỗ Như Tài, *Bài tập xấp xỉ hàm số dùng mạng MLP* và *Sample_Report*, tài liệu môn Học sâu, Trường Đại học Sài Gòn.
8. I. Sutskever, J. Martens, G. Dahl, G. Hinton, "On the importance of initialization and momentum in deep learning," *ICML*, 2013.
9. T. Tieleman, G. Hinton, "Lecture 6.5 – RMSProp," *COURSERA: Neural Networks for Machine Learning*, 2012.
10. T. Dozat, "Incorporating Nesterov Momentum into Adam," *ICLR Workshop*, 2016.
11. L. Prechelt, "Early Stopping — But When?" in *Neural Networks: Tricks of the Trade*, Springer, 1998.

Phụ lục A. Lệnh tái lập

Cài đặt: python -m venv .venv, kích hoạt môi trường, pip install -r requirements.txt, rồi pytest -q. Trên Windows đặt PYTHONUTF8=1. Thứ tự chạy (chi tiết trong README.md):

```
python src/generate_data.py                              # sinh data/*.csv
python src/data/eda.py                                   # thống kê, hình EDA
python src/run_experiment.py --config configs/grid_baseline.json
python src/run_experiment.py --config configs/grid_optimizer.json
python src/run_experiment.py --config configs/grid_batch.json
python src/run_experiment.py --config configs/expA.json       # tương tự: expA_steps, expB,
                                         # expB_wide, expB_A200, expC, expC_ctrl
python src/posthoc_test.py                               # hậu kiểm test trên best.pt
python src/analyze_stopping.py
python src/data/summarize_expA.py
python src/summarize_expB.py
python src/summarize_expC.py
python src/make_report_assets.py                         # chọn mô hình cuối, đánh giá test
python src/make_tables.py                                # bảng report/tables/
python src/run_experiment.py --config configs/demo.json       # demo khoảng 10 giây
```

Phụ lục B. Nhật ký quyết định

Bảng dưới tóm tắt report/NHAT_KY_QUYET_DINH.md (lý do và số liệu chi tiết của từng quyết định nằm trong file đó). Cột "Thứ tự" ghi rõ quyết định được khảo sát rồi mới chọn, hay chọn trước rồi kiểm chứng sau.

Bảng B.1. *Nhật ký quyết định.*

#	Quyết định	Thứ tự	Minh chứng
D1	Chia 2000/200/100, seed 2026/2027/2028	Chọn trước	data/meta.json
D2	Lấy x đều trên miền	Chọn trước	src/data/gaussian.py
D3	Tập train của A lồng nhau	Chọn trước	src/data/dataset.py::subset
D4	Scale x theo biên miền, y theo y_max	Chọn trước	src/data/gaussian.py
D5	sigmoid, lr 0.01	Khảo sát → chọn	results/grid_baseline/
D6	Adam	Chọn trước → kiểm chứng	results/grid_optimizer/
D7	Batch 32	Chọn trước → kiểm chứng	results/grid_batch/
D8	Patience 200	Chọn trước → kiểm chứng	results/stopping/patience_sim.csv
D9	max_epochs 2000	Chọn trước; hạn chế	results/stopping/epoch_cap.csv
D10	3 seed	Chọn trước; hạn chế	configs/*.json
D11	Đối chứng A cùng ngân sách bước	Phát hiện từ log	results/expA_steps/
D12	B mở rộng 16/32/64 và B ở A = 200	Phát hiện từ log	results/expB_wide/, results/expB_A200/
D13	Đối chứng C cùng số tham số	Chọn trước	results/expC_ctrl/
D14	Đo chuẩn gradient từng lớp	Chọn trước	results/expC/grad_norms.csv
D15	Mô hình cuối = argmin val trên 53 dòng của summary_all.csv	Khảo sát → chọn	results/summary_all.csv, configs/final.json
D16	Test chỉ dùng hậu kiểm và đánh giá cuối	Quy tắc	results/*/posthoc_test.log
D17	Tìm siêu tham số theo từng tọa độ	Chọn trước; hạn chế	—